

DOCKER — COMPLETE NOTES

From Containers 101 to Docker Compose & Best Practices

1. What is Docker?

Docker is an open-source platform that packages an application together with everything it needs to run — code, runtime, system libraries, and settings — into a single unit called a **container**. Because the container carries its own environment, it runs the same way on a developer's laptop, a test server, or in the cloud. This solves the classic *"it works on my machine"* problem.

Why Docker is used

- **Consistency** — same behaviour across dev, test, and production environments.
- **Isolation** — each container runs independently, without conflicting dependencies.
- **Lightweight** — containers share the host OS kernel, so they start in seconds and use far less resource than a VM.
- **Portability** — build once, run anywhere Docker is installed (Windows, macOS, Linux, cloud).
- **Scalability** — easy to spin up multiple identical containers for scaling and load balancing.
- **Faster CI/CD** — simplifies build, test, and deployment pipelines.

2. Docker vs Virtual Machines

A Virtual Machine virtualizes an entire computer, including its own OS kernel, making it heavy and slow to boot. A container virtualizes only the application layer and shares the host's OS kernel, making it lightweight and fast.

Aspect	Virtual Machine	Docker Container
OS	Full guest OS per VM	Shares host OS kernel
Size	GBs	MBs
Boot time	Minutes	Seconds
Performance	Overhead from hypervisor	Near-native performance
Isolation level	Strong (hardware-level)	Process-level isolation
Portability	Less portable	Highly portable

3. Docker Architecture

Docker follows a client–server architecture with three main components:

- **Docker Client** — the CLI (docker command) you type commands into.
- **Docker Daemon (dockerd)** — background service that builds, runs, and manages containers/images. Listens for API requests.

- **Docker Registry** — stores images. Docker Hub is the default public registry; private registries are also supported.

Flow: the client sends a command (e.g. `docker run`) to the daemon → the daemon pulls the image from a registry if not present locally → the daemon creates and starts a container from that image.

4. Core Concepts

Image

A read-only template with instructions for creating a container — it includes the application code, libraries, dependencies, and runtime. Images are built in layers, and each layer is cached for faster rebuilds.

Container

A running (or stopped) instance of an image. It's an isolated, executable process with its own filesystem, network, and process space — think of it as a lightweight, disposable "box" running your app.

Dockerfile

A plain text file containing step-by-step instructions to assemble a Docker image automatically.

Volume

A mechanism for persisting data generated by containers, independent of the container's lifecycle. Data in a container's writable layer is lost when the container is removed — volumes solve that.

Network

Docker provides virtual networks so containers can communicate with each other and with the outside world.

Docker Compose

A tool for defining and running multi-container applications using a single YAML file (`docker-compose.yml`).

5. Installation & First Steps

- Install **Docker Desktop** (Windows/macOS) or **Docker Engine** (Linux).
- Verify installation:

```
docker --version
docker run hello-world
```

The `hello-world` image confirms Docker can pull images and run containers correctly.

6. Essential Docker Commands (Cheat Sheet)

Images

Command	Description
<code>docker pull <image></code>	Download an image from a registry
<code>docker images</code>	List local images
<code>docker build -t <name> .</code>	Build an image from a Dockerfile in current dir
<code>docker rmi <image></code>	Remove an image
<code>docker tag <src> <target></code>	Tag an image (e.g. for pushing)
<code>docker push <image></code>	Push an image to a registry

Containers

Command	Description
<code>docker run <image></code>	Create and start a container
<code>docker run -d -p 8080:80 <image></code>	Run detached, map host:container port
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers (incl. stopped)
<code>docker stop <id/name></code>	Stop a running container
<code>docker start <id/name></code>	Start a stopped container
<code>docker restart <id/name></code>	Restart a container
<code>docker rm <id/name></code>	Remove a container
<code>docker exec -it <id> bash</code>	Open an interactive shell inside a running container
<code>docker logs <id/name></code>	View container logs
<code>docker inspect <id/name></code>	Show detailed container/image metadata

System & Cleanup

Command	Description
<code>docker system df</code>	Show disk usage by Docker

Command	Description
<code>docker system prune</code>	Remove unused data (stopped containers, dangling images, etc.)
<code>docker volume ls / rm</code>	List / remove volumes
<code>docker network ls</code>	List networks

7. Writing a Dockerfile

Common instructions

Instruction	Purpose
FROM	Base image to build on top of
WORKDIR	Sets the working directory inside the image
COPY / ADD	Copies files from host into the image
RUN	Executes a command at build time (e.g. install packages)
ENV	Sets environment variables
EXPOSE	Documents the port the container listens on
CMD	Default command to run when container starts (can be overridden)
ENTRYPOINT	Fixed command that always runs; args can be appended
ARG	Build-time variable
VOLUME	Declares a mount point for persistent data

Example — Node.js app

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install --production

COPY . .

EXPOSE 3000
CMD ["node", "server.js"]
```

Build and run it:

```
docker build -t my-node-app .
docker run -d -p 3000:3000 my-node-app
```

8. Volumes & Data Persistence

By default, data inside a container is lost once it's removed. Docker offers three ways to persist or share data:

Type	Description
Named Volume	Managed by Docker, stored in Docker's storage area. Best for persistent app data (e.g. databases).
Bind Mount	Maps a specific host directory/file into the container. Useful for local development.

Type	Description
tmpfs Mount	Stored in host memory only, never written to disk. Useful for sensitive/temp data (Linux only).

```
# Named volume
docker volume create mydata
docker run -v mydata:/app/data my-image

# Bind mount
docker run -v $(pwd):/app my-image
```

9. Docker Networking

Network Driver	Use Case
bridge (default)	Default network for standalone containers on a single host
host	Container shares the host's network stack directly (no isolation)
none	Disables networking entirely
overlay	Connects containers across multiple Docker hosts (Swarm)

10. Docker Compose

Docker Compose lets you define a multi-container application (e.g. a web app + database) in one YAML file and start everything with a single command.

Example — web app + Postgres

```
version: "3.9"
services:
  web:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - db
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - db-data:/var/lib/postgresql/data
volumes:
  db-data:
```

Command	Description
<code>docker compose up</code>	Build and start all services
<code>docker compose up -d</code>	Start in detached mode
<code>docker compose down</code>	Stop and remove containers, networks
<code>docker compose ps</code>	List services status
<code>docker compose logs -f</code>	Follow logs of all services

11. Multi-Stage Builds

Multi-stage builds keep final images small by using one stage to compile/build the app and a second, minimal stage to actually run it — build tools never end up in the final image.

```
# Stage 1: build
FROM node:20 AS build
WORKDIR /app
COPY . .
RUN npm install && npm run build

# Stage 2: run
FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

12. Best Practices

- Use small base images (e.g. alpine) to reduce size and attack surface.

- Use a `.dockerignore` file to exclude unnecessary files (`node_modules`, `.git`, etc.) from the build context.
- Order Dockerfile instructions so rarely-changing layers (like dependency installs) come before frequently-changing code — improves build cache usage.
- Never store secrets (passwords, API keys) inside images — use environment variables or secret managers.
- Use multi-stage builds to keep production images lean.
- Run containers as a non-root user where possible for security.
- Tag images meaningfully (avoid relying only on `:latest`).
- Regularly prune unused images/containers/volumes to save disk space.
- One process per container — keep containers focused and single-purpose.

13. Quick Recap

Term	One-line meaning
Image	Blueprint/template for a container
Container	Running instance of an image
Dockerfile	Recipe to build an image
Volume	Persistent storage outside container lifecycle
Registry	Storage/distribution service for images (e.g. Docker Hub)
Compose	Tool to run multi-container apps via YAML
Daemon	Background service that manages Docker objects

End of notes — practice by containerizing one of your own projects with a Dockerfile and a docker-compose.yml.